

注意力机制

注意力机制（英語：attention）是人工神经网络中一种模仿认知注意力的技术。这种机制可以增强神经网络输入数据中某些部分的权重，同时减弱其他部分的权重，以此将网络的关注点聚焦于数据中最重要的一小部分。数据中哪些部分比其他部分更重要取决于上下文。可以通过梯度下降法对注意力机制进行训练。

类似于注意力机制的架构最早于1990年代提出，当时提出的名称包括乘法模块（multiplicative module）、sigma pi 单元、超网络（hypernetwork）等。^[1]注意力机制的灵活性来自于它的“软权重”特性，即这种权重是可以在运行时改变的，而非像通常的权重一样必须在运行时保持固定。注意力机制的用途包括神经图灵机中的记忆功能、可微分神经计算机中的推理任务^[2]、Transformer 模型中的语言处理、Perceiver（感知器）模型中的多模态数据处理（声音、图像、视频和文本）。^{[3][4][5][6]}

概述

假设我们有一个以索引 ***i*** 排列的标记（token）序列。对于每一个标记 ***i***，神经网络计算出一个相应的满足

∑

i

w

i

=
1

{\displaystyle \sum _{i}w_{i}=1}

 的非负软权重 ***w_i***。每个标记都对应一个由词嵌入得到的向量 ***v_i***。加权平均

∑

i

w

i

v

i

{\displaystyle \sum _{i}w_{i}v_{i}}

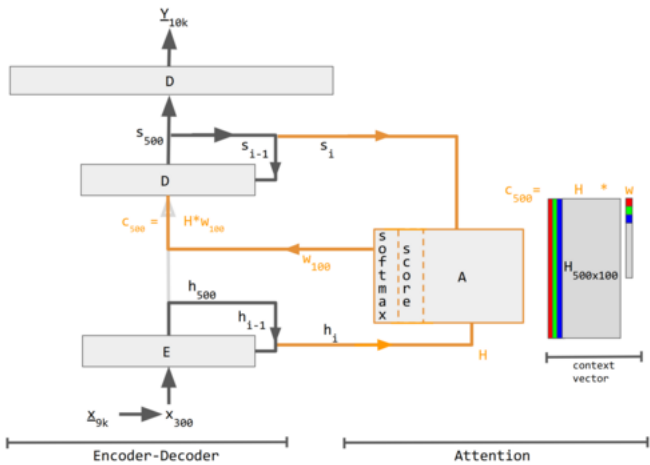
 即是注意力机制的输出结果。

可以使用查询-键机制（query-key mechanism）计算软权重。从每个标记的词嵌入，我们计算其对应的查询向量 ***q_i*** 和键向量 ***k_i***。再计算点积 ***q_ik_j*** 的 softmax 函数便可以得到对应的权重，其中 ***i*** 代表当前标记、***j*** 表示与当前标记产生注意力关系的标记。

某些架构中会采用多头注意力机制（multi-head attention），其中每一部分都有独立的查询（query）、键（key）和值（value）。

语言翻译示例

下图展示了将英语翻译成法语的机器，其基本架构为编码器-解码器结构，另外再加上了一个注意力单元。在图示的简单情况下，注意力单元只是循环层状态的点积计算，并不需要训练。但在实践中，注意力单元由需要训练的三个完全连接的神经网络层组成。这三层分别被称为查询（query）、键（key）和值（value）。



加入注意力机制的编码器-解码器架构。图中使用具体的数值表示向量的大小，使其更为直观。左侧黑色箭头表示的是编码器-解码器，中间橘色箭头表示的是注意力单元，右侧灰色与彩色方块表示的是计算的数据。矩阵H与向量w中的灰色区域表示零值。数值下标表示向量大小。字母下标i与i-1表示计算步。

图例	
标签	描述
100	语句最大长度
300	嵌入尺寸（词维度）
500	隐向量长度
9k, 10k	输入、输出语言的词典大小
$\underline{x}, \underline{y}$	大小为 9k 与 10k 的独热词典向量。 $\underline{x} \rightarrow x$ 以查找表实现。 $\underline{y}$ 是解码器 D 线性输出的 argmax 值。
x	大小为 300 的词嵌入向量，通常使用 GloVe 或 word2vec 等模型预先计算得到的结果。
h	大小为 500 的编码器隐向量。对于每一计算步，该向量包含了之前所有出现过的词语的信息。最终得到的 h 可以被看作是一个“句”向量，杰弗里·辛顿则称之为“思维向量”（thought vector）。
s	大小为 500 的解码器隐向量。
E	500 个神经元的循环神经网络编码器。输出大小为 500。输入大小为 800，其中 300 为词嵌入维度，500 为循环连接。编码器仅在初始化时直接连接到解码器，故箭头以淡灰色表示。
D	两层解码器。循环层有 500 个神经元，线性全连接层则有 10k 个神经元（目标词典大小）。 ^[7] 单线性层就包含 500 万（ $500 \times 10k$ ）个参数，约为循环层参数的 10 倍。
score	大小为 100 的对准分数
w	大小为 100 的注意力权重向量。这些权重为“软”权重，即可以在前向传播时改变，而非只在训练阶段改变的神经元权重。
A	注意力模块，可以是循环状态的点积，也可以是查询-键-值全连接层。输出是大小为 100 的向量 w。
H	500×100 的矩阵，即 100 个隐向量 h 连接而成的矩阵。
c	大小为 500 的上下文向量 $= H * w$ ，即以 w 对所有 h 向量取加权平均。

下表是每一步计算的示例。为清楚起见，表中使用了具体的数值或图形而非字母表示向量与矩阵。嵌套的图形代表了每个 h 都包含之前所有单词的历史记录。在这里，我们引入注意力分数以得到所需的注意力权重。

步	x	h, H = 编码器输出 大小为 500×1 的向量，以图 形表示	s = 解码器提供给注意力单元的输入	对准分数	w = 注意力权重 = softmax(分数)	c = 上下文向量 = H*w	y = 解码器输出
1	I	 = “I” 的向量编码	-	-	-	-	-
2	love	 = “I love” 的向量编码	-	-	-	-	-
3	you	 = “I love you” 的向量 编码	-	-	-	-	-
4	-	-	解码器尚未初始化，故使用编码器输出 h ₃ 对其初始化 	[.63 -.32 -.25 .5 .5 ...]	[.94 .02 .04 0 0 ...]	.94 *  + .02 *  + .04 * 	je
5	-	-	s ₄	[-1.5 -3.9 .57 .5 .5 ...]	[.11 .01 .88 0 0 ...]	.11 *  + .01 *  + .88 * 	t'
6	-	-	s ₅	[-2.8 .64 -3.2 .5 .5 ...]	[.03 .95 .02 0 0 ...]	.03 *  + .95 *  + .02 * 	aime

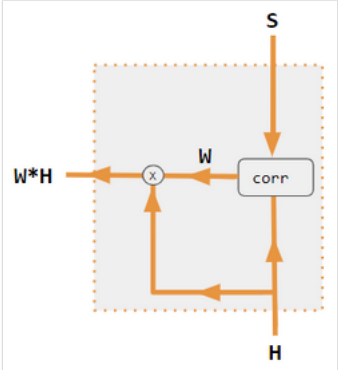
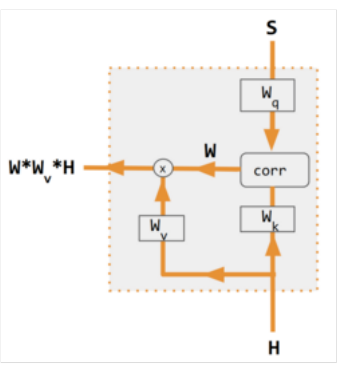
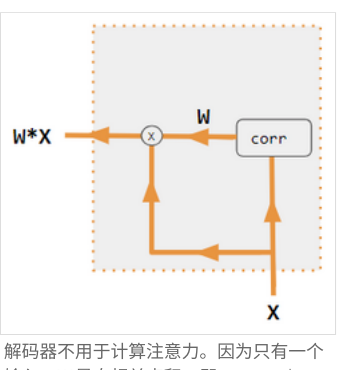
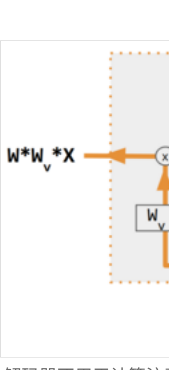
以矩阵展示的注意力权重表现了网络如何根据上下文调整其关注点。

	I	love	you
je			
t'			
aime			

对注意力权重的这种展现方式回应了人们经常用来批评神经网络的可解释性问题。对于一个只作逐字翻译而不考虑词序的网络，其注意力权重矩阵会是一个对角占优矩阵。这里非对角占优的特性表明注意力机制能捕捉到更为细微的特征。在第一次通过解码器时，94%的注意力权重在第一个英文单词“I”上，因此网络的输出为对应的法语单词“je”（我）。而在第二次通过解码器时，此时88%的注意力权重在第三个英文单词“you”上，因此网络输出了对应的法语“t'”（你）。最后一遍时，95%的注意力权重在第二个英文单词“love”上，所以网络最后输出的是法语单词“aime”（爱）。

变体

注意力机制有许多变体：点积注意力（dot-product attention）、QKV 注意力（query-key-value attention）、强注意力（hard attention）、软注意力（soft attention）、自注意力（self attention）、交叉注意力（cross attention）、Luong 注意力、Bahdanau 注意力、稀疏注意力^[8]等。这些变体重新组合编码器端的输入，以将注意力效果重新分配到每个目标输出。通常而言，由点积得到的相关式矩阵提供了重新加权系数（参见图例）。

1. 编码器-解码器点积	2. 编解码器 QKV	3. 编码器点积	4. 编码器
 同时需要编码器与解码器来计算注意力。^[9]	 同时需要编码器与解码器来计算注意力。^[10]	 解码器不用于计算注意力。因为只有一个输入，W是自相关点积，即 $w_{ij} = x_i * x_j$。^[11]	 解码器不用于计算注意力。

图例

标签	描述
变量 X,H,S,T	大写变量代表整句语句，而不仅仅是当前单词。例如，H 代表编码器隐状态的矩阵——每列代表一个单词。
S, T	S = 解码器隐状态，T = 目标词嵌入。在 Pytorch 示例变体训练阶段，T 在两个源之间交替，具体取决于所使用的教师强制（teacher forcing）级别。T 可以是网络输出词的嵌入，即 embedding(argmax(FC output))。或者当使用教师强制进行训练时，T 可以是已知正确单词的嵌入。可以指定其发生的概率（如 1/2）。
X, H	H = 编码器隐状态，X = 输入词嵌入
W	注意力系数
Qw, Kw, Vw, FC	分别用于查询、键、向量的权重矩阵。FC 是一个全连接的权重矩阵。
带圈+，带圈x	带圈+ = 向量串联。带圈x = 矩阵乘法
corr	逐列取 softmax(点积矩阵)。点积在变体 3 中的定义是 $\mathbf{x}_i \cdot \mathbf{x}_j$ ，在变体 1 中是 $\mathbf{h}_i \cdot \mathbf{s}_j$ ，在变体 2 中是 列 _i (Kw*H) * 列 _j (Qw*S)，在变体 4 中是 列 _i (Kw*X) * 列 _j (Qw*X)。变体 5 则使用全连接层来确定系数。对于 QKV 变体，则点积由 sqrt(d) 归一化，其中 d 是 QKV 矩阵的高度。

参考文献

1. Yann Lecun. [Deep Learning course at NYU, Spring 2020, video lecture Week 6](#). 事件发生在 53:00. 2020 [2022-03-08]. （原始内容存档于 2023-07-09）.

2. Graves, Alex; Wayne, Greg; Reynolds, Malcolm; Harley, Tim; Danihelka, Ivo; Grabska-Barwińska, Agnieszka; Colmenarejo, Sergio Gómez; Grefenstette, Edward; Ramalho, Tiago; Agapiou, John; Badia, Adrià Puigdomènech; Hermann, Karl Moritz; Zwols, Yori; Ostrovski, Georg; Cain, Adam; King, Helen; Summerfield, Christopher; Blunsom, Phil; Kavukcuoglu, Koray; Hassabis, Demis. [Hybrid computing using a neural network with dynamic external memory](#). Nature. 2016-10-12, **538** (7626): 471–476 [2022-06-07]. Bibcode:2016Natur.538..471G. ISSN 1476-4687. PMID 27732574. S2CID 205251479. doi:10.1038/nature20101. （原始内容存档于 2022-10-02）（英语）.

3. Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N.; Kaiser, Lukasz; Polosukhin, Illia. [Attention Is All You Need](#). 2017-12-05. arXiv:1706.03762 [cs.CL].

4. Ramachandran, Prajit; Parmar, Niki; Vaswani, Ashish; Bello, Irwan; Levskaya, Anselm; Shlens, Jonathon. [Stand-Alone Self-Attention in Vision Models](#). 2019-06-13. arXiv:1906.05909 [cs.CV].

5. Jaegle, Andrew; Gimeno, Felix; Brock, Andrew; Zisserman, Andrew; Vinyals, Oriol; Carreira, Joao. [Perceiver: General Perception with Iterative Attention](#). 2021-06-22. arXiv:2103.03206 [cs.CV].

6. Ray, Tiernan. [Google's Supermodel: DeepMind Perceiver is a step on the road to an AI machine that could process anything and everything](#). ZDNet. [2021-08-19]. （原始内容存档于 2021-10-29）（英语）.

7. [Pytorch.org seq2seq tutorial](#). [December 2, 2021].

8. Yuan, Jingyang; Gao, Huazuo; Dai, Damai; Luo, Junyu; Zhao, Liang; Zhang, Zhengyan; Xie, Zhenda; Wei, Y. X.; Wang, Lean, [Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention](#), 2025-02-27 [2026-03-07], doi:10.48550/arXiv.2502.11089

9. Luong, Minh-Thang. [Effective Approaches to Attention-based Neural Machine Translation](#). 2015-09-20. arXiv:1508.04025v5 [cs.CL].

10. Neil Rhodes. [CS 152 NN—27: Attention: Keys, Queries, & Values](#). 事件发生在 06:30. 2021 [2021-12-22]. （原始内容存档于 2022-10-21）.

11. Alfredo Canziani & Yann Lecun. [NYU Deep Learning course, Spring 2020](#). 事件发生在 05:30. 2021 [2021-12-22]. （原始内容存档于 2023-03-22）.

12. Alfredo Canziani & Yann Lecun. [NYU Deep Learning course, Spring 2020](#). 事件发生在 20:15. 2021 [2021-12-22]. （原始内容存档于 2023-03-22）.

13. Robertson, Sean. [NLP From Scratch: Translation With a Sequence To Sequence Network and Attention](#). pytorch.org. [2021-12-22].

取自 “<https://zh.wikipedia.org/w/index.php?title=注意力机制&oldid=91795615>”